

Chapter 5: Data Binding

1. Overview

Data binding is the mechanism Angular uses to synchronize data between a component's TypeScript class (the "model") and its template (the "view"). It is the backbone of Angular's declarative UI programming model: instead of manually querying the DOM and setting properties or attaching listeners (as you would with vanilla JS or jQuery), you declare *bindings* in the template, and Angular's change detection and rendering pipeline keeps the DOM in sync with the component state.

Angular supports four conceptual binding categories:

1. **Interpolation** (`{{ expression }}`) — text interpolation into the DOM.
2. **Property binding** (`[property]="expression"`) — one-way, from component to DOM element property, directive input, or component `@Input()`.
3. **Event binding** (`(event)="statement"`) — one-way, from DOM/component event to component method.
4. **Two-way binding** (`[(ngModel)]="expression"` or `[(x)]="expression"`) — a combination of property + event binding sugar, most commonly seen with forms via `ngModel`, or custom via the `x / xChange` convention.

A recurring interview theme is that **all of these are really just syntactic sugar over property and event bindings**, which themselves compile down to specific Angular "instructions" (`ɵɵproperty`, `ɵɵlistener`, `ɵɵtextInterpolate`, etc.) in the Ivy compiler output. Understanding this compilation model, the DOM property-vs-attribute distinction, and how change detection decides *when* to push new values into the DOM is what separates "I use Angular" from "I understand Angular" at interview level.

This chapter also covers how binding syntax and semantics evolve with Angular Signals — the reactive primitive introduced in v16+ that changes how and when bindings read values, without changing the binding *syntax* itself.

2. Core Concepts

2.1 Interpolation

```
<h1>{{ title }}</h1>
<p>Total: {{ price * quantity }}</p>
```

- Interpolation is **one-way**: component → view.
- The expression inside `{{ }}` is an Angular template expression, not arbitrary JavaScript. It supports a restricted subset: property access, method calls, ternary, `&&/||`, template literals (limited), the safe navigation operator `?.`, and pipes (`|`). It does **not** support assignments, `new`, `++/--`, bitwise operators, or `,` (comma) operators, and it cannot throw uncaught — Angular catches and logs template expression errors without crashing the app (in dev mode it surfaces as `ExpressionChangedAfterItHasBeenCheckedError` or console errors depending on context).
- Interpolation is syntactic sugar for a **property binding to `textContent`** conceptually, but under the

hood it compiles to dedicated interpolation instructions (`@textInterpolate1`, etc. — see §4.3), not a generic property binding.

- Interpolation can appear inside attribute values too: `` — but this is discouraged; Angular internally rewrites it to a property binding `[src]`. For anything other than plain text nodes, prefer explicit property binding.

2.2 Property Binding

```
<img [src]="imageUrl">
<button [disabled]="isSaving">Save</button>
<app-user-card [user]="currentUser"></app-user-card>
```

- Syntax: `[target]="expression"`.
- `target` can be:
 - A **DOM element property** (`src`, `disabled`, `value`, `textContent`, `hidden`, ...)
 - A **directive/component** `@Input()` (`[user]`, `[ngClass]`, `[formControl]`)
 - A **synthetic property** for animations (`[@myTrigger]`)
- Property binding sets the **DOM property**, not the HTML attribute (critical distinction — see §2.4 and §4.2).
- One-way: component → view only. If the expression's value changes, Angular updates the DOM property on the next change detection pass. Changes to the DOM property itself (e.g., a user typing in an `<input>`, which changes `input.value`) do **not** flow back to the component automatically — that's why `<input [value]="name">` alone does not update `name` as the user types.
- Angular allows binding target names to be written with or without brackets for **string literal attributes** with no brackets, e.g. `<div class="static"></div>` (a plain attribute, evaluated once, not a binding). The brackets are what tell the compiler "treat the right side as an expression, not a literal string."

2.3 Event Binding

```
<button (click)="save()">Save</button>
<input (keyup.enter)="submit()">
<app-child (itemSelected)="onItemSelected($event)"></app-child>
```

- Syntax: `(target)="statement"`.
- `target` is a DOM event name (`click`, `input`, `keyup`) or a custom `@Output()` `EventEmitter` name on a directive/component.
- The right-hand side is a **template statement**, not an expression — it *can* have side effects (assignments, method calls, chained statements with `;`).
- `$event` is a special implicit variable holding the event payload:
 - For native DOM events, `$event` is the native `Event` (or subtype like `KeyboardEvent`, `MouseEvent`).
 - For custom `@Output()` bindings, `$event` is whatever value was passed to `.emit(value)`.
- **Event modifiers/pseudo-events** like `keyup.enter`, `click.stop` (in newer Angular template syntax

with event modifiers, or historically via `$event.stopPropagation()`), and filtering keys (`keydown.control.z`) are handled specially by Angular's event plugin system (`KeyEventsPlugin`).

- Event bindings are registered once (typically) and Angular wraps the handler to trigger change detection afterward (in Zone.js-based apps, Zone.js itself patches `addEventListener` so *any* DOM event triggers a check; Angular's event binding wraps your listener and, after running it, may also call `markForCheck` / rely on zone re-entry, and in zoneless apps explicitly schedules a CD pass).

2.4 Attribute Binding

```
<td [attr.colspan]="colspanValue"></td>
<button [attr.aria-pressed]="isPressed"></button>
<div [attr.data-id]="item.id"></div>
```

- Syntax: `[attr.name]="expression"`.
- Used when there is **no corresponding DOM property** for the thing you want to set — most commonly ARIA attributes (`aria-*`), `colspan` / `rowspan` on table cells, `role`, custom `data-*` attributes, and SVG attributes (many SVG attributes have no property reflection).
- Setting an attribute binding to `null` or `undefined` **removes the attribute** from the DOM entirely (unlike property binding, which just sets the property to a falsy value while the attribute — if it existed statically — remains).
- This is the #1 interview trap: `colspan` has no DOM property equivalent on `HTMLTableCellElement` in the way Angular's property binder expects, so `[colspan]="2"` silently fails (or, depending on Angular version/dev mode, warns), while `[attr.colspan]="2"` works correctly.

2.5 Class and Style Bindings

Single-class binding:

```
<div [class.active]="isActive"></div>
<div [class.disabled]="!isEnabled"></div>
```

Multi-class binding (object or string):

```
<div [class]=" 'foo bar' "></div>
<div [class]="{ active: isActive, disabled: !isEnabled }"></div>
<div [ngClass]="{ active: isActive, disabled: !isEnabled }"></div>
```

Single-style binding (with optional unit):

```
<div [style.width.px]="widthValue"></div>
<div [style.color]="isError ? 'red' : 'black' "></div>
```

Multi-style binding (object):

```
<div [style]="{ width: widthValue + 'px', color: textColor }"></div>
<div [ngStyle]="{ width: widthValue + 'px' }"></div>
```

- `[class.x]` and `[style.x]` are Angular's built-in **host/class/style binding syntax**, distinct from the `NgClass` / `NgStyle` directives. Since Ivy, `[class]` / `[style]` / `[class.x]` / `[style.x]` are compiled with special, highly optimized instructions (`ɵɵclassProp`, `ɵɵstyleProp`, `ɵɵclassMap`, `ɵɵstyleMap`) and are generally **preferred over** `NgClass` / `NgStyle` for performance — they avoid the directive's `ngDoCheck`-based diffing and integrate directly into style/class "binding slots" on the element, with Angular merging static classes/styles, host bindings from directives, and template bindings at compile time via a priority/specificity algorithm.
- `NgClass` / `NgStyle` are still valid and more convenient for dynamic key sets, but incur extra overhead (they are directives with their own change-detection hooks).
- Multiple `[class.x]` bindings and a `[class]` map binding on the same element **compose** rather than conflict — Ivy resolves them at compile time based on binding order and specificity (more specific single-property bindings win over map bindings by default, but the resolution order and Angular version can matter for edge cases — this is one of Ivy's "style binding precedence" internals worth knowing at a senior level).

2.6 Two-Way Binding — Banana in a Box

```
<input [(ngModel)]="username">
```

- The `[( )]` syntax is nicknamed **"banana in a box"** (`(` looks like a banana inside `[ ]`, the box).
- It is pure **syntactic sugar** — Angular desugars it at compile time into:

```
<input [ngModel]="username" (ngModelChange)="username = $event">
```

- General rule: `[(x)]` requires the target directive/component to expose **both**:
 - An `@Input() x` (property binding)
 - An `@Output() xChange` — an `EventEmitter` that emits the new value
- For `ngModel` specifically, `FormsModule`'s `NgModel` directive implements this contract: it reads the initial value via `[ngModel]`, listens to the native `input` / `change` events on the host control, and emits `ngModelChange` with the updated value.
- **Custom two-way binding example:**

```
@Component({
  selector: 'app-counter',
  template: `<button (click)="increment()">{{ count }}</button>`
})
export class CounterComponent {
  @Input() count = 0;
  @Output() countChange = new EventEmitter<number>();

  increment() {
    this.count++;
    this.countChange.emit(this.count);
  }
}
```

```
<app-counter [(count)]="parentCount"></app-counter>
```

- With Angular v17.1+ **signal-based two-way binding**, `model()` replaces the `@Input()` / `@Output()` pair entirely:

```
@Component({ selector: 'app-counter', template: `<button (click)="increment()">{{ count() }}</button>` })
export class CounterComponent {
  count = model(0); // creates both a readable input signal and an implicit `countChange` output
  increment() { this.count.update(v => v + 1); }
}
```

`[(count)]="parentCount"` still works unchanged from the consumer's perspective — the banana-in-a-box syntax is agnostic to whether the underlying mechanism is decorator-based `@Input` / `@Output` or `model()`.

2.7 Binding to Directives and Components

- Property binding targets are resolved by Angular's compiler against the **static template type-checking metadata** (in strict template mode) by matching `[name]` against a directive's/component's declared `@Input()` (or `input()` signal) with that public name (or its alias).
- Multiple directives can be applied to the same host element, and each can expose its own inputs/outputs — Angular disambiguates purely by input/output name, not by directive identity, so name collisions between directives on the same element are a real hazard.
- Binding to a plain DOM element with no matching directive input falls back to a raw DOM property binding; if neither a directive input nor a DOM property matches the name, Angular throws `NG0304: 'x' is not a known property of '<tag>'` at compile/runtime (in strict/full template type-checking, this is a compile-time error).

2.8 Bindings and Signals

Angular Signals (stable since v17) do not introduce new binding *syntax* — they change **what you put on the right-hand side** and, more importantly, **how/when the binding is refreshed**.

```
<p>{{ fullName() }}</p>
<input [value]="username()">
<button [disabled]="isSaving()">Save</button>
```

Key differences vs. plain property/observable-based bindings:

- A signal is read by **calling it** (`mySignal()`), which registers that specific template binding as a **consumer/subscriber** of that signal within Angular's reactive graph.
- Traditional (non-signal) bindings are refreshed by **Zone.js-driven, tree-wide change detection**: any async event that Zone.js patches (click, `setTimeout`, `fetch`, etc.) triggers Angular to walk the component tree from the root (or from the dirty-marked branch), re-evaluating every binding expression to see if it changed (dirty-checking).
- **Signal-based bindings are granular**. When a signal used in a template changes, Angular knows exactly which bindings/consumers depend on it and can, in principle, update just those bindings without re-

running the full component's change detection function tree-wide — this is the foundation for **zoneless Angular** (`provideZonelessChangeDetection()`), where there is no Zone.js at all, and change detection is scheduled *only* when a signal read in a template changes (via `markForCheck` -equivalent notification from the signal's dependency graph), not on every async browser event.

- Practically, this means with signals: `[disabled]="isSaving()"` re-evaluates when `isSaving` signal's value changes and Angular schedules a check for the component; with a plain field `[disabled]="isSaving"`, Angular re-evaluates the binding on *every* CD pass (any tick), regardless of whether `isSaving` actually changed, and merely diffs the previous vs. new value before touching the DOM.
- Interpolation and property binding both work identically syntactically whether the RHS is a signal call, a plain property, or an `| async`-pipable observable — this uniformity is intentional so that migrating from Zone-based/RxJS-based state to Signals doesn't require template rewrites beyond adding `()`.
- Event bindings are unaffected by signals — `(click)="save()"` is the same regardless of whether `save()` internally reads/writes signals or plain fields. Signals only change the *read* side (property binding/interpolation), not the *write*/event side.
- Two-way binding: `model()` (see §2.6) is the signal-native replacement for `@Input()` / `@Output()` pairs, giving you a `WritableSignal`-like object under the hood usable with `[(x)]`.

3. Code Examples

3.1 Interpolation vs. Property Binding for the Same Value

```
<!-- Interpolation: works, but only for text content -->
<p>{{ user.name }}</p>

<!-- Property binding: required for non-text-content properties -->
<input [value]="user.name">
<img [src]="user.avatarUrl" [alt]="user.name">
```

3.2 Attribute vs. Property — the Classic Gotcha

```
<!-- WRONG: colspan has no DOM property; this does nothing on some Angular versions or warns -->
<td [colspan]="span"></td>

<!-- CORRECT: use attribute binding -->
<td [attr.colspan]="span"></td>

<!-- disabled DOES have a DOM property, so property binding is correct here -->
<button [disabled]="isSaving">Save</button>
```

```

@Component({
  selector: 'app-row',
  template: `<td [attr.colspan]="span">{{ label }}</td>`
})
export class RowComponent {
  @Input() span = 1;
  @Input() label = '';
}

```

3.3 Two-Way Binding, Manual Desugared Form

```

<!-- Sugar -->
<input [(ngModel)]="email" name="email">

<!-- Fully equivalent, desugared -->
<input [ngModel]="email" (ngModelChange)="email = $event" name="email">

```

3.4 Custom Two-Way Binding (Classic Decorator Style)

```

@Component({
  selector: 'app-rating',
  template: `
    <button *ngFor="let s of stars" (click)="select(s)">
      {{ s <= value ? '★' : '☆' }}
    </button>
  `
})
export class RatingComponent {
  @Input() value = 0;
  @Output() valueChange = new EventEmitter<number>();
  stars = [1, 2, 3, 4, 5];

  select(star: number) {
    this.value = star;
    this.valueChange.emit(this.value);
  }
}

```

```

<app-rating [(value)]="productRating"></app-rating>

```

3.5 Custom Two-Way Binding with `model()` (Signals)

```
@Component({
  selector: 'app-rating',
  template: `
    @for (s of stars; track s) {
      <button (click)="select(s)">{{ s <= value() ? '★' : '☆' }}</button>
    }
  `
})
export class RatingComponent {
  value = model(0);
  stars = [1, 2, 3, 4, 5];
  select(star: number) { this.value.set(star); }
}
```

3.6 Class/Style Binding Combinations

```
<div
  class="card"
  [class.card--selected]="isSelected"
  [class.card--disabled]="!isEnabled"
  [style.borderColor]="isSelected ? '#0a84ff' : 'transparent'"
  [style.opacity]="isEnabled ? 1 : 0.5">
  {{ item.name }}
</div>
```

3.7 Event Binding with `$event` and Modifiers

```
<input
  (keyup.enter)="submit()"
  (input)="onInput($event)"
  (blur)="onBlur($event.target.value)">
```

```
onInput(event: Event) {
  const value = (event.target as HTMLInputElement).value;
  this.query.set(value);
}
```

3.8 Signal-Driven Property Binding vs. Plain Field

```
@Component({
  selector: 'app-status',
  template: `
    <p>Plain field: {{ plainCount }}</p>
    <p>Signal: {{ signalCount() }}</p>
    <button (click)="incrementBoth()">Increment</button>
  `
})
```



```
export class StatusComponent {
  plainCount = 0;
  signalCount = signal(0);

  incrementBoth() {
    this.plainCount++;
    this.signalCount.update(v => v + 1);
  }
}
```

Both update visually the same way here because a click event still triggers full CD (Zone.js present) — the *practical* difference in update mechanics only becomes visible in zoneless apps or with `onPush` + non-signal mutations that don't call `markForCheck`.

4. Internal Working

4.1 Compilation: Templates Become Instructions

Angular's Ivy compiler (`@angular/compiler`) transforms every component template into a **template function** made of low-level instruction calls, executed in two conceptual phases per component instance:

- **Create mode** (runs once per instance): builds the DOM structure — `ɵɵelementStart`, `ɵɵtext`, `ɵɵelementEnd`, `ɵɵlistener` (registers event handlers).
- **Update mode** (runs on every change detection pass for that component): re-evaluates binding expressions and pushes new values into the DOM — `ɵɵadvance`, `ɵɵproperty`, `ɵɵattribute`, `ɵɵclassProp`, `ɵɵstyleProp`, `ɵɵtextInterpolate1`.

Example — `<button [disabled]="issaving">{{ label }}</button>` compiles roughly to:

```
// create mode
ɵɵelementStart(0, 'button');
ɵɵlistener(...);
ɵɵtext(1);
ɵɵelementEnd();

// update mode (runs every CD tick for this component)
ɵɵadvance(0);
ɵɵproperty('disabled', ctx.issaving);
ɵɵadvance(1);
ɵɵtextInterpolate1(ctx.label);
```

Each instruction (`ɵɵproperty`, etc.) performs an internal **"binding index" dirty check**: it compares the newly computed value against the value stored from the previous run (`bindingupdated()` in Ivy internals). Only if the value actually differs (using `!==`, i.e., reference/primitive equality — not deep equality) does Angular call down into the renderer (`Renderer2`/native DOM API) to actually mutate the DOM. This is why binding a new object literal or array literal inline in a template (`[data]="{a: 1}"`) is a classic performance footgun — it creates a new reference every CD run, so the "did it change" check always says yes, causing unnecessary downstream work (e.g., re-running `onPush` children, `ngonchanges` firing every tick).

4.2 DOM Property vs. HTML Attribute

This distinction is foundational and frequently tested:

- An **HTML attribute** is what's written in the markup/HTML source (`<input value="abc">`) and is exposed via `Element.getAttribute() / setAttribute()`. Attributes are always strings.
- A **DOM property** is a field on the JavaScript object representing that element (`inputEl.value`), which can be any JS type (string, boolean, number, object) and reflects the element's *live, current* state.
- For most standard attributes, the browser creates an initial property value from the attribute (attribute → property reflection at parse time), but after that they diverge: typing into an `<input>` changes the `value` **property** immediately; the `value` **attribute** stays frozen at whatever the initial HTML had (that's why `getAttribute('value')` after user typing still shows the original markup value while `.value` shows what's on screen).
- Angular's `[property]="expr"` binding **always sets the DOM property**, calling the equivalent of `element.property = value` via the renderer — never `setAttribute`. This is why it correctly handles booleans/objects and why it can't be used for attributes that have no property counterpart.
- Angular's `[attr.name]="expr"` binding **always calls** `setAttribute / removeAttribute`, stringifying the value (or removing the attribute entirely when the value is `null / undefined`).
- Static, non-bound attributes in the template (`<div id="app">`) are just HTML attributes set once at element creation — no ongoing binding exists for them.

4.3 Interpolation Internals

`{{ expr }}` is not literally `[textContent]="expr"`. Ivy has dedicated, arity-optimized instructions: `ɵɵtextInterpolate`, `ɵɵtextInterpolate1` ... `ɵɵtextInterpolate8`, and `ɵɵtextInterpolatev` for more than 8 expressions — each specialized for the number of dynamic expressions embedded in the interpolated string, avoiding generic string-concatenation overhead and array allocation for the common cases. This micro-optimization (avoiding a generic "build array of parts, join" for the 99% case of 1–8 interpolated values) is a good example of Ivy's general philosophy: generate the most specific, allocation-minimal instruction possible per binding site.

4.4 Event Binding Internals

- `(click)="handler($event)"` compiles to `ɵɵlistener('click', function ($event) { return ctx.handler($event); })` registered during create mode via the renderer's `listen()` API (which, in a browser, is `addEventListener` — potentially wrapped by Zone.js's monkey-patched version).
- The listener closure captures the component context (`ctx`) so it can call back into instance methods.
- After the handler returns, Angular (in Zone-based apps) relies on `NgZone` having detected the async task boundary (the event handler itself runs *inside* the Angular zone because Zone.js patched `addEventListener`), and schedules a change detection pass (`tick()`) once the zone reports "stable" (no more microtasks/macrotasks pending from that event). In zoneless apps, Angular explicitly calls `markForCheck` / schedules a CD pass right after any listener created via `ɵɵlistener` runs, and after signal writes.

4.5 Two-Way Binding Internals

`[(ngModel)]="x"` desugars purely at the **template compiler** level (a syntax transform, no runtime magic beyond what property/event bindings already do) into `[(ngModel)]="x" (ngModelChange)="x = $event"`. There is no special "two-way binding instruction" in Ivy — it's literally two instructions: a `ɵɵproperty` call for `ngModel` and a `ɵɵlistener` call for `ngModelChange`, wired to the same template context field. This is provable by inspecting the compiled output (`ng build` with source maps, or the Angular compiler's AST tools) — you will find no distinct "two-way" instruction, only the pair.

4.6 Class/Style Binding Internals & Precedence

`[class.foo]`, `[class]`, `[style.foo]`, and `[style]` all lower to `ɵɵclassProp`, `ɵɵclassMap`, `ɵɵstyleProp`, `ɵɵstyleMap` respectively. Ivy maintains **per-element "styling contexts"** that merge:

1. Static `class` / `style` attributes on the element.
2. Host bindings for class/style contributed by every directive on the element (including the component itself).
3. Template-level class/style bindings.

At runtime, these are resolved via a defined priority (more specific — single property — bindings generally take precedence over map bindings, and template bindings can override host bindings depending on Angular version's exact algorithm) so the final `className` / `style` written to the DOM is a single consistent merge rather than each directive fighting to call `setAttribute('class', ...)` independently — which is precisely why the old `Renderer2`-only, non-Ivy approach to manual class manipulation was more error-prone.

4.7 Change Detection's Role — Why "One-Way" Bindings Update at All

Property/interpolation bindings are re-evaluated every time Angular runs change detection for that component. Change detection is triggered by:

- Zone.js noticing an async browser API completed (event, timer, XHR/fetch) — the classic Zone-based trigger.
- An `AsyncPipe` receiving a new emission from an Observable/Promise it's subscribed to (calls `markForCheck` internally).
- A signal read in a template changing value (in modern Angular, this notifies the reactive graph, which schedules a check for the owning component, independent of Zone.js).
- Manual calls: `ChangeDetectorRef.detectChanges()` / `markForCheck()`, or `ApplicationRef.tick()`.

During a check, Angular walks down from the root (or, with `OnPush` components, skips subtrees whose components haven't been marked dirty and have no new `@Input()` references or signal changes) and re-runs each component's update-mode instructions, which is where the individual `ɵɵproperty` / `ɵɵtextInterpolate` dirty-checks (§4.1) happen.

5. Edge Cases & Gotchas

1. `[colspan]` vs `[attr.colspan]` — no DOM property for `colspan` in the way Angular expects; must

use `[attr.colspan]`. Same category of issue applies to `rowspan`, ARIA attributes (`[attr.aria-label]`, though note `ariaLabel` *does* exist as a property on some elements in modern browsers — but `[attr.aria-*)` remains the safe, portable idiom), and most SVG presentation attributes (`[attr.fill]`, `[attr.d]`).

- Boolean attributes look "set" even when false.** `<button disabled="false">` is still disabled in raw HTML because attribute presence, not its string value, matters — but `[disabled]="false"` (property binding) correctly leaves the button enabled, because Angular sets the actual boolean property. This is a strong argument for always preferring property binding over string attribute literals for boolean HTML attributes.
- ngModel requires a name attribute inside <form>.** Using `[(ngModel)]` on a control inside a template-driven `<form>` without a `name` attribute throws a runtime error (NG01350 -style: "must have a name attribute") because Angular auto-registers it as a form control keyed by that name — unless you opt out with `[ngModelOptions]="{standalone: true}"`.
- [(ngModel)] on a component you wrote yourself won't work unless you implement ControlValueAccessor or expose the value / valueChange @Input() / @Output() pair (or model()).** Simply naming your inputs/outputs `ngModel / ngModelChange` does not hook into Angular Forms' `NgModel` directive machinery.
- New object/array/function literals in bindings break OnPush and cause churn.** `[config]="{ a: 1 }"` or `(click)="doThing(items.filter(...))"` creates a new reference every CD run — defeats `OnPush`'s reference-equality skip logic and can cause `ngOnChanges` to fire every check on the child even though "nothing really changed." Prefer memoized/stable references, computed signals (`computed()`), or precomputed class fields.
- Interpolation inside attributes (`src="{{ url }}"`) is legal but discouraged** — prefer `[src]="url"` directly; the interpolated-attribute form is rewritten by Angular internally but is less explicit and doesn't work at all for non-string-coercible bindings.
- Two-way binding naming convention is mandatory and case-sensitive.** For `[(x)]` to work on a custom directive, the output **must** be named exactly `xchange` (not `xchanged`, not `onxchange`). Angular's compiler statically checks for this exact suffix when desugaring `[(x)]`; if `xchange` doesn't exist, you get a template compile error (in strict mode) or the binding silently only applies the input half.
- \$event type is any by default outside of strict template checking**, but with `strictTemplates: true` (the modern default in `tsconfig`), Angular infers `$event`'s type from the DOM event map (`HTMLElementEventMap`) for native events, and from the `EventEmitter<T>` generic for custom outputs — giving you real compile-time type checking on event handlers. This is a good reason to always enable `strictTemplates`.
- Property binding doesn't "unbind" cleanly to a falsy DOM default the way attribute removal does.** `[attr.data-x]="null"` removes the attribute; `[value]="null"` on an `<input>` typically coerces to the empty string or `"null"` depending on the property's setter semantics, not attribute removal — because you're setting a JS property, and `null / undefined` handling is property-specific, not a uniform Angular behavior.
- ExpressionChangedAfterItHasBeenCheckedError (NG0100)** commonly bites developers who mutate bound state (often inside `ngAfterViewInit` / child lifecycle hooks) after the parent has already been checked in the same CD cycle, because a bound expression's value differs between two consecutive checks within one detection pass — a direct consequence of the per-binding value comparison described in §4.1.

11. **Signals inside `@Input()`-decorated properties do not make the input itself reactive/granular** unless you specifically use the `input()` signal function (Angular v17.1+) — a plain `@Input() foo: signal<number>` field still relies on the *reference* to the signal object being passed down, and reads (`foo()`) inside the template are what actually establish reactivity; the `@Input` mechanism itself is unrelated to signal reactivity.
 12. **`[class]` / `[style]` map bindings overwrite classes/styles you set with a raw static `class="..."` attribute if not merged carefully in older Angular versions** — Ivy's merging model (§4.6) resolves this in modern versions, but it's a known source of "my static class disappeared" bugs historically and worth mentioning defensively.
-

6. Interview Questions & Answers

Q1. What are the four types of data binding in Angular, and which direction does data flow for each?

A: (1) Interpolation `{{ }}` — one-way, component → view. (2) Property binding `[prop]` — one-way, component → view. (3) Event binding `(event)` — one-way, view → component. (4) Two-way binding `[(x)]` — combines property + event binding, so it flows both ways: component → view (via the input) and view → component (via the output emitting new values back).

Q2. What's the difference between an HTML attribute and a DOM property, and why does Angular care?

Interviewer intent: Checks whether the candidate understands binding at the browser level, not just Angular syntax — a common gap that causes real bugs (e.g., `colspan`).

A: An attribute is markup text parsed from HTML, always a string, accessed via `getAttribute` / `setAttribute`; it typically only initializes the corresponding DOM property once. A DOM property is a live JS object field that can hold any type and reflects current state (e.g., `input.value` changes as the user types, while the `value` attribute stays at its initial HTML value). Angular's `[prop]` binding sets the DOM property directly (via the renderer, not `setAttribute`), which is why it must be used for anything dynamic and type-rich (booleans, objects), while `[attr.name]` explicitly calls `setAttribute` / `removeAttribute` and is required for attributes with no property counterpart (`colspan`, ARIA attributes, SVG attributes).

Q3. Why does `<td [colspan]="2">` not work as expected, and what's the fix?

A: `HTMLTableCellElement` doesn't have Angular's expected simple settable `colspan` DOM property binding path the way `disabled` or `value` do — practically, Angular's `[colspan]` doesn't map correctly, and the safe/correct approach is `[attr.colspan]="2"`, which uses `setAttribute('colspan', '2')` directly. General rule: if there's no reliable DOM property for what you want, use attribute binding.

Q4. What does `[(ngModel)]="x"` desugar into, and what two things must a custom component expose to support the same `[(x)]` syntax?

A: It desugars to `[(ngModel)]="x" (ngModelChange)="x = $event"`. For a custom `[(x)]` binding to work, the component/directive must expose an `@Input() x` and an `@Output() xChange` (an `EventEmitter`) — the naming convention `xChange` is mandatory and exact. As of Angular v17.1+, `model()` provides both halves automatically without manually declaring the `@Output()`.

Q5. Why is `[(ngModel)]` on an unnamed control inside a template-driven form an error?

A: Inside a `<form>`, `NgModel` auto-registers itself with Angular's `NgForm` / `FormGroupDirective` machinery keyed by the control's `name` attribute so the form can track/validate it. Without a `name`, Angular can't register the control and throws an error (unless `[(ngModelOptions)]="{standalone: true}"` opts the control out of form registration).

Q6. Explain how Angular decides whether to actually touch the DOM when a property binding's expression is re-evaluated.

Interviewer intent: Tests understanding of Ivy's per-binding "dirty check" mechanism and its performance implications — a strong signal of someone who has looked past surface syntax.

A: Every property/attribute/class/style/interpolation binding compiles to an Ivy instruction (`ɵɵproperty`, `ɵɵattribute`, etc.) that runs on every change-detection pass for that component. Internally, each binding "slot" stores the previous value; the instruction compares the newly computed value to the stored one using `!==` (reference/primitive equality, not deep equality). Only on an actual difference does Angular call into the renderer to mutate the real DOM. This means binding to a freshly-created object/array/function literal every render (`[cfg]="{a:1}"`) always looks "changed" and causes unnecessary DOM writes and downstream re-checks, which is why stable references / memoization matter, especially with `OnPush`.

Q7. What is `$event` and how does its type get inferred in event bindings?

A: `$event` is an implicit template variable holding the event payload — the native DOM `Event` (or subtype) for DOM event bindings, or whatever value was passed to `.emit()` for custom `@Output()` bindings. With `strictTemplates: true` in `tsconfig.json`, Angular's template type checker infers its type: from the `HTMLElementEventMap` (so `(keyup)` gives `KeyboardEvent`, `(click)` gives `MouseEvent`) for native events, and from the generic type parameter of `EventEmitter<T>` for custom outputs.

Q8. How do class and style bindings (`[class.x]`, `[style.x]`) differ from `NgClass` / `NgStyle`, and which should you prefer?

A: `[class.x]` / `[style.x]` / `[class]` / `[style]` are Angular's built-in binding syntax, compiled directly to dedicated Ivy instructions (`ɵɵclassProp`, `ɵɵstyleProp`, `ɵɵclassMap`, `ɵɵstyleMap`) that integrate into the element's styling context alongside static classes/styles and directive host bindings, resolved with defined precedence — no extra directive machinery involved. `NgClass` / `NgStyle` are structural directives with their own `ngDoCheck`-based diffing logic, which is more flexible for dynamic key sets computed at runtime but carries extra per-check overhead. General guidance: prefer the built-in `[class.x]` / `[style.x]` syntax for known, fixed sets of classes/styles; reach for `NgClass` / `NgStyle` when the set of active classes/styles is genuinely dynamic/computed as a map.

Q9. Why might a static `class="card"` attribute plus `[class.selected]="isSelected"` behave unexpectedly in older Angular apps, and how does Ivy fix this?

A: In pre-Ivy Angular (View Engine), interactions between static classes, directive host class bindings, and template class bindings could conflict or overwrite each other because they weren't uniformly merged. Ivy introduced per-element "styling contexts" that merge static attributes, all directive host class/style contributions, and template-level class/style bindings into one consistent final `class` / `style` value applied to the DOM, with a defined precedence order, eliminating most of these conflicts.

Q10. How do signals change the way property bindings are updated compared to traditional Zone.js-based change detection?

Interviewer intent: Distinguishes candidates who've only used signals syntactically from those who understand the underlying scheduling/reactivity shift, especially relevant for zoneless Angular questions.

A: Syntactically nothing changes — you still write `[disabled]="isSaving()"` or `{{ count() }}`. The difference is *how updates are scheduled and how much work happens*. With Zone.js, any patched async browser API firing (click, timer, HTTP) triggers a full change-detection pass that walks the tree and re-evaluates every binding expression's instruction to diff old vs. new values, regardless of whether the underlying data actually changed. With signals, reading a signal in a template (`mySignal()`) registers that binding as a dependent in Angular's reactive graph; when the signal's value changes, Angular is told precisely which components/bindings depend on it and schedules a check only for those — this is what enables zoneless

change detection (`provideZonelessChangeDetection()`), where there's no Zone.js at all and updates are driven purely by signal writes (and a few other explicit triggers like `markForCheck`), rather than "any async event happened, recheck everything."

Q11. Can you two-way bind directly to a signal without `model()` , e.g. `[(ngModel)]="mySignal"` ?

A: No — not directly, because `[(ngModel)]="mySignal"` desugars to `[(ngModel)]="mySignal" (ngModelChange)="mySignal = $event"` , and assigning to a signal variable (`mySignal = $event`) doesn't call `.set()` ; it would just reassign the local variable/field reference, not update the underlying signal's value, and TypeScript would likely flag the type mismatch anyway (assigning a raw value to a `WritableSignal<T>`-typed field). You'd need `[(ngModel)]="mySignal()" (ngModelChange)="mySignal.set($event)"` , or better, use `model()` for genuinely signal-native two-way binding, which is designed to work correctly with `[(x)]` syntax out of the box.

Q12. What causes NG0100: ExpressionChangedAfterItHasBeenCheckedError , and how does it relate to how bindings are evaluated?

A: It fires (in dev mode) when a bound expression's value differs between two consecutive change-detection checks within the same overall detection pass — e.g., a parent component's template reads `child.someValue` , CD checks it once, then a lifecycle hook like `ngAfterViewInit` or a child's own change detection mutates the state the parent already read, and Angular's second verification pass (dev-mode-only "check no changes" run) notices the discrepancy. It's a direct consequence of the fact that every binding's evaluated value is compared and expected to be stable within one synchronous CD cycle — the fix is usually to move the mutation earlier (e.g., `ngOnInit`), defer it a tick (`setTimeout / Promise.resolve().then()`), or explicitly call `detectChanges()` /use signals, which handle this more gracefully because their propagation model differs from ad hoc field mutation.

Q13. Why is binding a method call directly in a template, e.g. `[(value)]="computeValue()"` , considered risky for performance?

A: Because that expression re-executes on every change-detection check for that component (potentially many times per user interaction, since Angular re-runs update-mode instructions on every CD pass, not just when relevant inputs change), so any nontrivial computation, especially one that allocates new objects/arrays, becomes both a performance cost and a change-tracking hazard (§Edge Case 5). Prefer precomputed fields, memoized getters guarded by actual dependency changes, or — in modern Angular — `computed()` signals, which only recompute when their tracked dependencies actually change and cache the result between reads.

Q14. What's the difference between a template *expression* (used in interpolation/property binding) and a template *statement* (used in event binding)?

A: Template expressions (interpolation, property/attribute/class/style bindings) are meant to be side-effect-free, read-only computations — Angular restricts the grammar (no assignments, `new` , increment/decrement, bitwise ops, comma operator) and evaluates them defensively (won't throw uncaught, supports safe-navigation `?.`). Template statements (event bindings) are expected to have side effects and support a richer grammar including assignments (`x = $event`) and statement chaining with `;` — because their entire purpose is to trigger state changes in response to user/DOM events, not to compute a displayable value.

7. Quick Revision Cheat Sheet

Syntax	Name	Direction	Compiles to (roughly)	Use for
<code>{{ expr }}</code>	Interpolation	Component → View	<code>ɵɵtextInterpolate*</code>	Text content

Syntax	Name	Direction	Compiles to (roughly)	Use for
<code>[prop]="expr"</code>	Property binding	Component → View	<code>ɵproperty</code> (sets DOM property)	DOM properties, <code>@Input()</code> s
<code>[attr.name]="expr"</code>	Attribute binding	Component → View	<code>ɵattribute</code> (<code>setAttribute</code> / <code>removeAttribute</code>)	No property equivalent: <code>colspan</code> , <code>aria-*</code> , SVG attrs
<code>(event)="stmt"</code>	Event binding	View → Component	<code>ɵlistener</code> (<code>addEventListener</code>)	DOM events, <code>@Output()</code> s
<code>[(x)]="expr"</code>	Two-way binding	Both	Sugar for <code>[x]="expr" (xChange)="expr = \$event"</code>	Forms (<code>ngModel</code>), custom <code>model()</code> / <code>@Input</code> + <code>@Output</code> pairs
<code>[class.x]</code> / <code>[style.x]</code>	Single class/style binding	Component → View	<code>ɵclassProp</code> / <code>ɵstyleProp</code>	Toggle one class / set one style prop
<code>[class]</code> / <code>[style]</code>	Class/style map binding	Component → View	<code>ɵclassMap</code> / <code>ɵstyleMap</code>	Multiple classes/styles from an object/string
<code>[ngClass]</code> / <code>[ngStyle]</code>	Directive-based class/style	Component → View	Directive with <code>ngOnChanges</code> diffing	Highly dynamic key sets (more overhead)

Golden rules:

- Property binding sets a **DOM property** (live, typed, JS-side). Attribute binding sets an **HTML attribute** (string, `setAttribute` / `removeAttribute`) — use it whenever no DOM property exists for what you're setting.
- `null` / `undefined` on `[attr.x]` **removes** the attribute; on `[prop]` it just sets the property to that value (property-specific coercion).
- Two-way binding is always sugar: `[(x)] ⇔ [x]="expr" (xChange)="expr = $event"`; custom components need `@Input() x` + `@Output() xChange` (or `model()`).
- Every property/interpolation binding is re-evaluated on every CD pass for that component and diffed with `!==` before touching the DOM — avoid literal object/array/function creation inline in bindings.
- `[class.x]` / `[style.x]` are compiled Ivy instructions, generally cheaper than `ngClass` / `ngStyle` for fixed/known toggles.
- Signals don't change binding syntax; they change *scheduling granularity* — a signal read in a template registers as a graph dependency, enabling targeted updates and zoneless change detection, versus Zone.js's "any async event → recheck everything" model.
- `model()` (v17.1+) is the signal-native equivalent of an `@Input()` / `@Output()` `xChange` pair for building your own two-way-bindable components.
- Template expressions (bindings) must be side-effect-free and use a restricted grammar; template statements (event bindings) allow assignments and side effects.

Created By - Durgesh Singh